PROJECT TWO: DISTRIBUTED LOCK DESIGN

TA: 赵桐 (Tong Zhao)

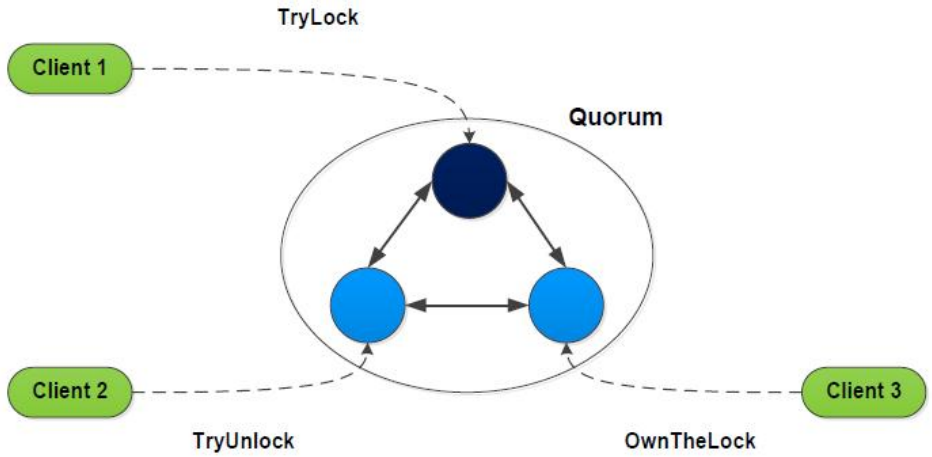
Mail: tongzhao@sjtu.edu.en

Project Web: http://www.cs.sjtu.edu.cn/~wuct/bdpt/project.html

Dead Line: 2025.11.30

PROJECT TWO

- Design a simple consensus system, which satisfies the following requirements,
  - Contain one leader server and multiple follower server
  - Each follower server has a replicated map, the map is consisted with the leader server. The key of map is the name of distributed lock, and the value is the Client ID who owns the distributed lock.



PROJECT TWO

- Support multiple clients to preempt/release a distributed lock, and check the owner of a distributed lock.
  - For preempting a distributed lock
    - -- If the lock doesn't exist, preempt success;
    - -- Otherwise, preempt fail;
  - For releasing a distributed lock
    - -- If the client owns the lock, release success;
    - -- Otherwise, release fail;
  - For checking a distributed lock
    - -- Any client can check the owner of a distributed lock

PROJECT TWO

- To ensure the data consistency of the system, the follower servers send all preempt/release requests to the leader server.
- To check the owner of a distributed lock, the follower server accesses its map directly and sends the results to the clients.
- When the leader server handling preempt/release requests:
  - If needed, modify its map and sends a request propose to all follower servers
  - When a follower server receives a request propose
    - -- modify its local map
    - -- check the request is pending or not
    - -- if the request is pending, send an answer to the client

PLATFORM

- Three Linux PC, Ubuntu system is suggested
- You can also use virtual machines on Windows instead.
- VMWARE and Virtual Box are suggested.
- https://www.virtualbox.org
- https://www.vmware.com

EXAMPLE

- You can use any kind of Programming language like C++,JAVA,PYTHON and so on.
- JAVA may be a better choice and I give a simple example here.

PREPARATORY WORK

- Java must be installed. Recommended Java versions are described at https://wiki.apache.org/hadoop/HadoopJavaVersions
- And the website to download JAVA is https://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html
- Put JAVA under /opt/
- tar –zxvf ./java/name /opt/
- Modify ~/.bashrc, add "**export export JAVA_HOME=/opt/yourjavaname**"

CONNECT

- Use Socket to connect client and server with each other.
- Many open source code can be found on the Internet

CLIENT

- For the client class, it contains static information such as the server ip and client ID corresponding to the client; it also contains the operations and specific implementations that the client can perform. There are three operations, and the functions are defined as follows:
- void tryLock (String lockName, String lockKey), to perform the locking operation;
- void tryUnLock (String lockName, String lockKey), to release the lock operation;
- String ownTheLock (String lockName, String lockKey), returns the slave of the lock.

CLIENT

- When the client needs to preempt the lock, releasing the lock and query the slave of the lock, the steps are the same:
  - sending the request message to the corresponding server
  - waiting for the return message of the receiving server to be presented to the client after processing.
- all three operations are implemented by
  - sendMsg (String msg): send the message
  - ReceiveMsg():receives the message.
- They send request messages to the corresponding server through the ip and port number bound by the client. The message includes the client ID, the request type (lock/unlock/ownTheLock), and the name of the request lock. Then wait for the server to return success, failure or lock object information. After receiving the return message and the client ends the operation, disconnect from the server.

SERVER

- The server class contains the server ip, whether it is the main server, the lock dictionary and other static information. Functions are defined as follows:
  - void newThread (String newip), responsible for opening a new thread to receive messages;
  - void connectLeader (String info), notify the main server;
  - void inform (String tmpIp, String info), synchronized from the server.

SERVER

- The server will always listen on the specified port. When the server finds a new client to send a message to it, it will open a new thread to communicate with the client through this thread.
- If the server is the primary server, when it receives the operation request,
  - first checks whether the operation is legal, including whether the command is correct, whether the lock exists in the dictionary, whether the owner of the lock is the client, and so on.
  - If the operation is legal, the primary server will modify the dictionary and then notify all slaves to perform dictionary modification.

SERVER

- If the server is a slave server, when an operation request is received,
  - First check the validity of the operation.
  - For legitimate operations, the slave server forwards the request to the primary server; the primary server checks again whether the operation conflicts. If everything is correct, the dictionary is modified accordingly, and then the slave server is notified; when receiving the message from the primary server, the slave server modifies dictionary directly to maintain consistency with the primary server.

RESULTS

```
Please input your command:
lock data1
lock successfully
after command, own the data1: ea30f970-07f5-412f-abf3-1c54e27749e2

Please input your command:
unlock data1
unlock successfully
after command, own the data1: lock not exist

Please input your command:
ownTheLock data1
own the data1: lock not exist

Please input your command:
lock data2
lock successfully
after command, own the data2: ea30f970-07f5-412f-abf3-1c54e27749e2
```